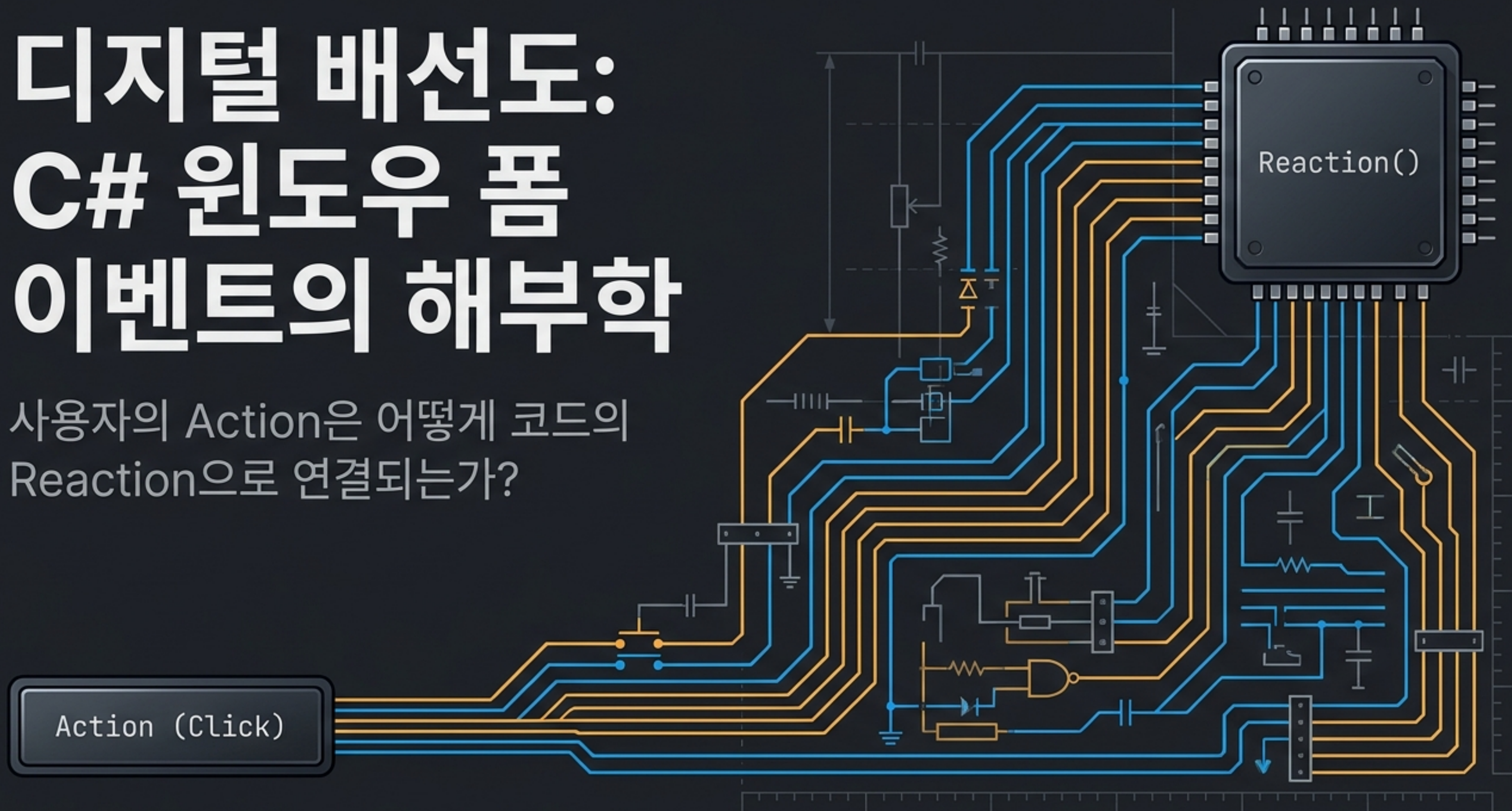
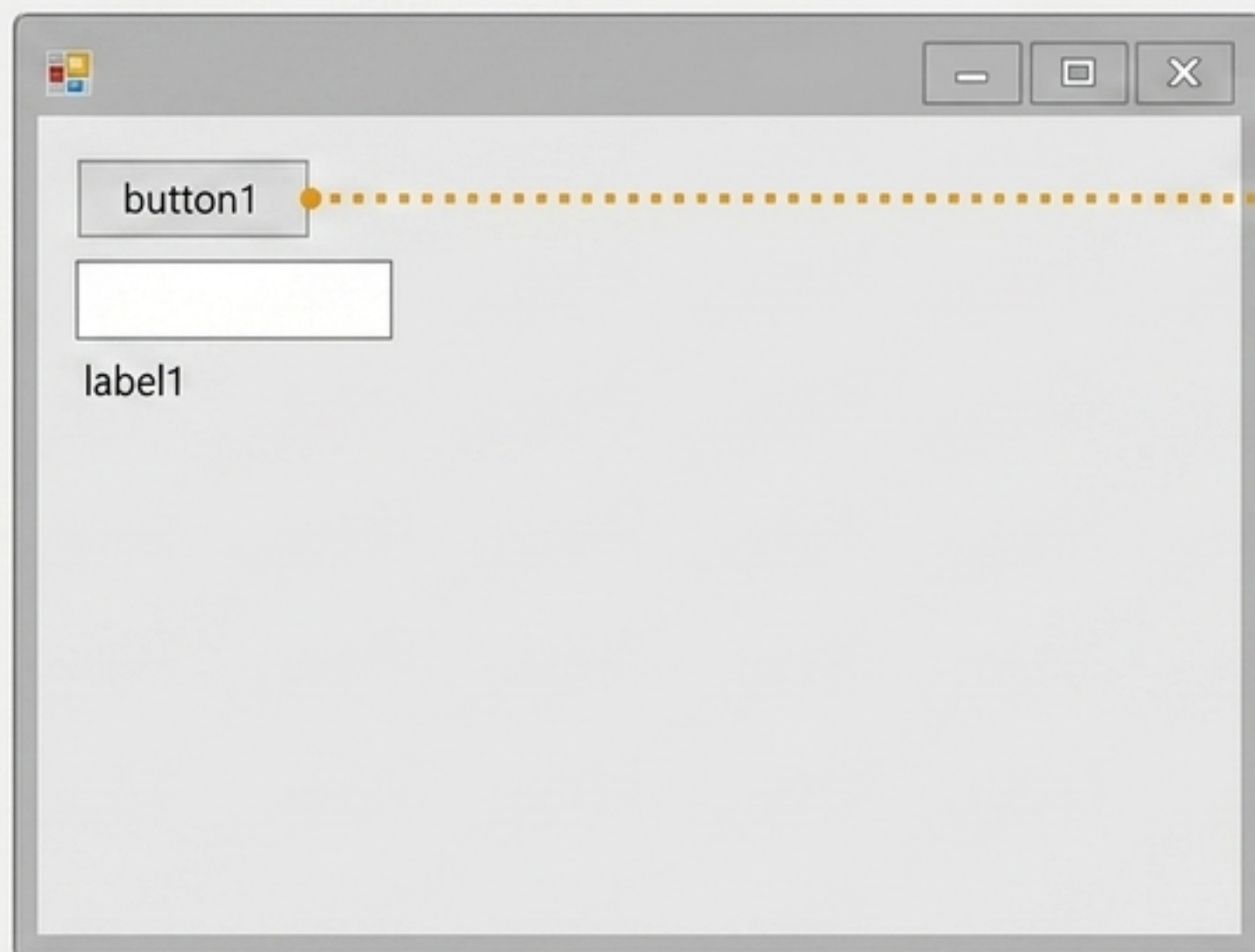


디지털 배선도: C# 윈도우 폼 이벤트의 해부학

사용자의 Action은 어떻게 코드의
Reaction으로 연결되는가?



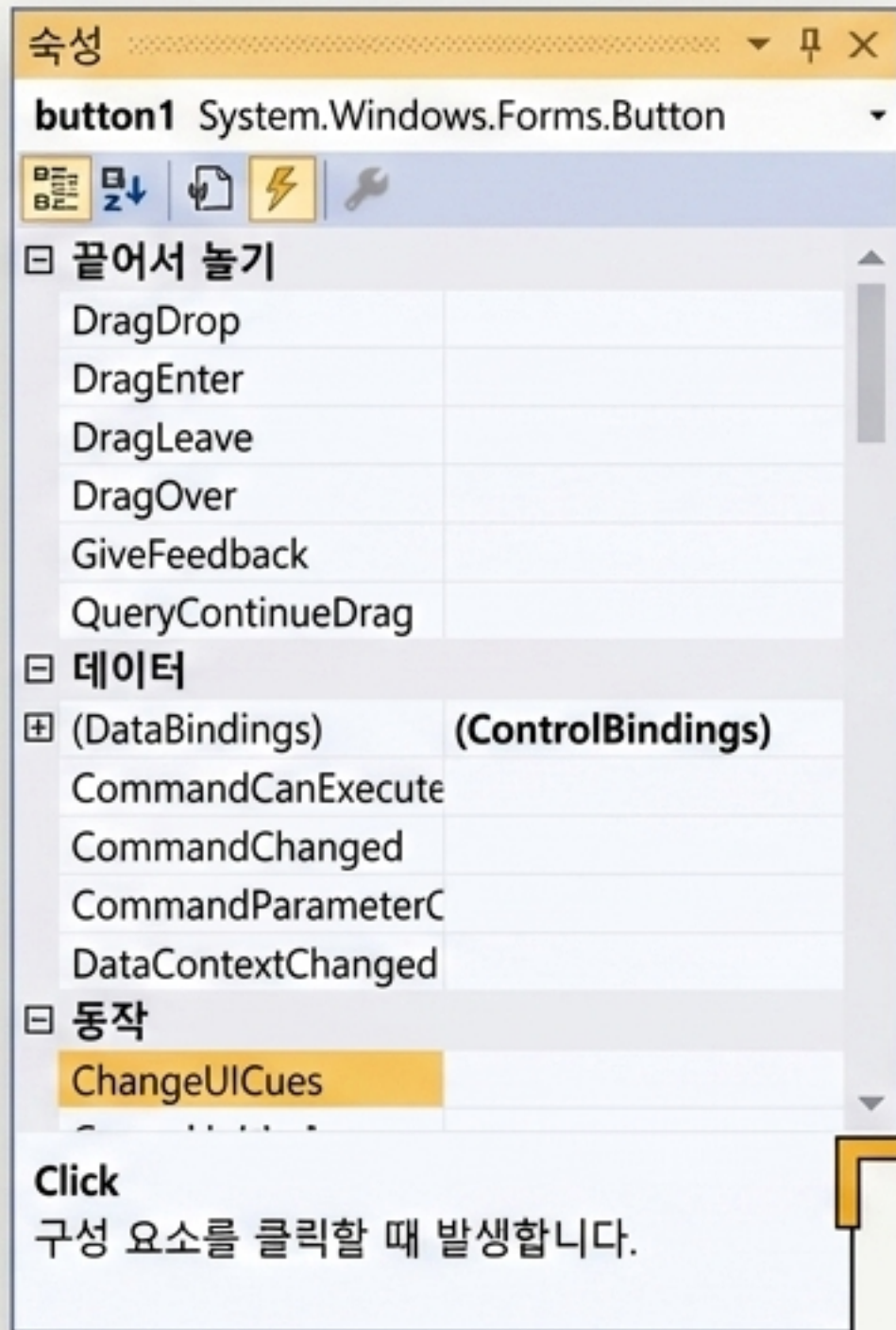
정적 UI (The Canvas)



목표 (The Goal)

**버튼을 눌렀을 때
무슨 일이 일어나게
할 것인가?**

UI 디자인은 껍데기에 불과합니다.
생명을 불어넣기 위해서는 사용자의
행동(Action)과 프로그램의 반응(Reaction)을
연결하는 배선 작업이 필요합니다.

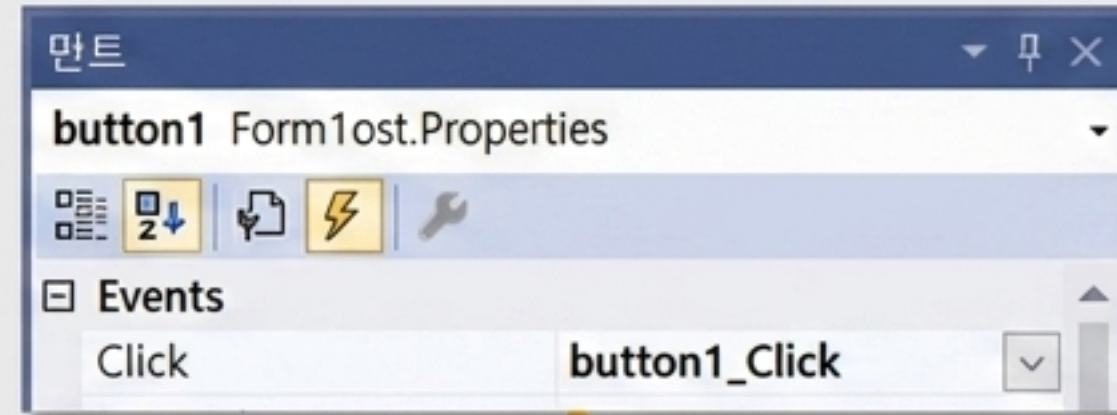


```
private Button button1;  
private TextBox textBox1;  
private Label label1;
```

번개 ⚡ = 이벤트(Event)

버튼을 더블클릭하는 순간, Visual Studio는 첫 번째 디지털 배선을 자동으로 연결합니다.

GUI의 설정이...



```
01 public partial class Form1 : Form
02 {
03     public Form1()
04     {
05         InitializeComponent();
06     }
07
08     private void button1_Click(object? sender, EventArgs e)
09     {
10
11     }
12 }
```

...코드의 생성으로 직결됨



이벤트 연결이 완료되면, Action은 즉각적으로 C# 메서드(Reaction)를 호출합니다.

button1

```
01 button1.Location = new Point(12, 12);  
02 button1.Name = "button1";  
03 button1.Size = new Size(75, 23);  
04 button1.TabIndex = 0;  
05 button1.Text = "button1";  
06 button1.UseVisualStyleBackColor = true;  
07 button1.Click += button1_Click;
```

button1.Click += button1_Click;

자동화의 장막 뒤: GUI 디자이너가 번개 아이콘을 통해 한 일은, 사실 이 한 줄의 코드를 작성한 것입니다.

+= 연산자: 이벤트(Click)에 메서드(button1_Click)를 물리적으로 접합하는 핵심 배선.

동적 연결 (Dynamic Wiring): 코드로 직접 제어하기

코딩 시작

```
1 public Form1()  
2 {  
3     InitializeComponent();  
4     button1.Click += |  
5 }
```

- Click
- DoubleClick
- MouseDown
- MouseDoubleClick
- PerformClick

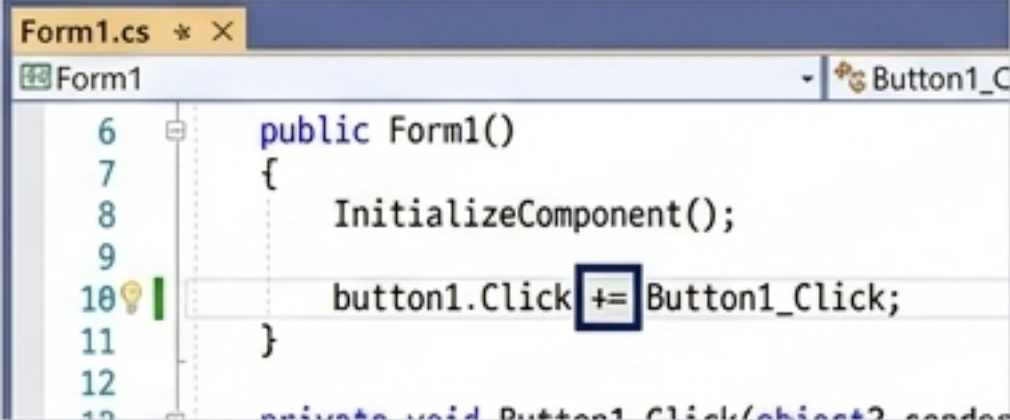


자동 완성 후

```
1 public Form1()  
2 {  
3     InitializeComponent();  
4     button1.Click += Button1_Click;  
5 }
```

Button1_Click; (삽입하려면 <Tab> 키 누름)

개발자가 원할 때 언제든지 코드를 통해 이벤트를 뗐다 붙였다(+=, -=) 할 수 있는
가장 강력하고 유연한 제어 방식입니다.

	정적 연결 (GUI Approach)	동적 연결 (Code Approach)
Iconography		
특징 - Characteristics	직관적이고 빠름 (초보자 친화적)	런타임에 유연한 제어 가능 (조건에 따라 -= 로 해제 가능)
코드 위치 - Code Location	Form1.Designer.cs 에 숨겨짐	개발자가 Form1.cs 등 원하는 곳에 직접 작성

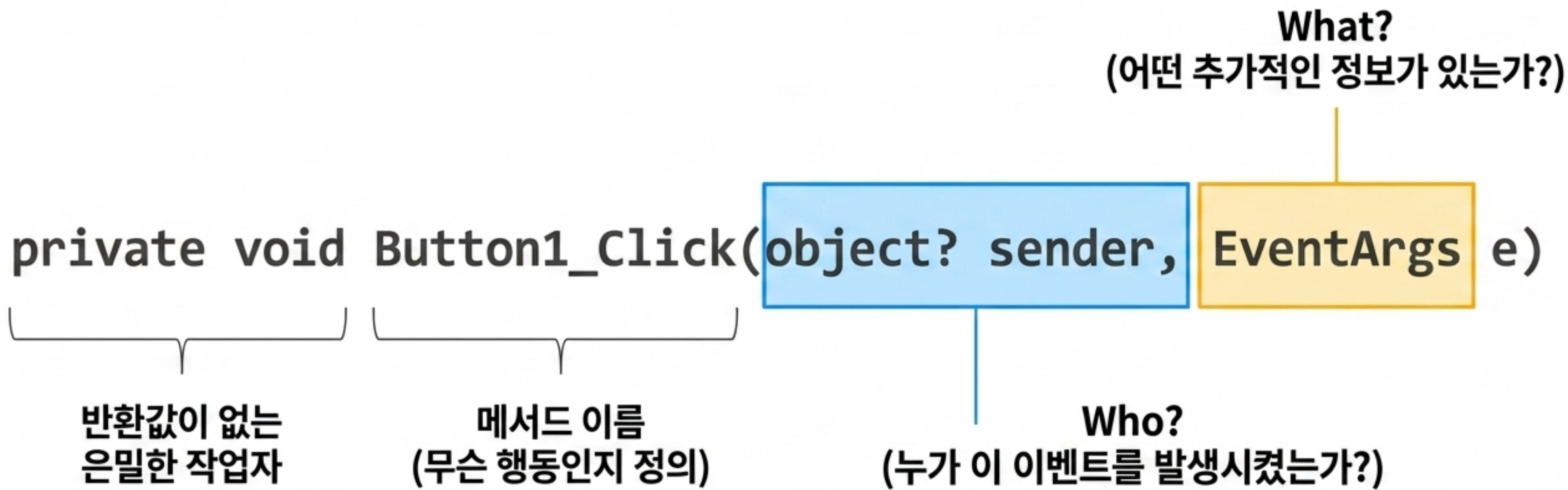
! Tip Box

```

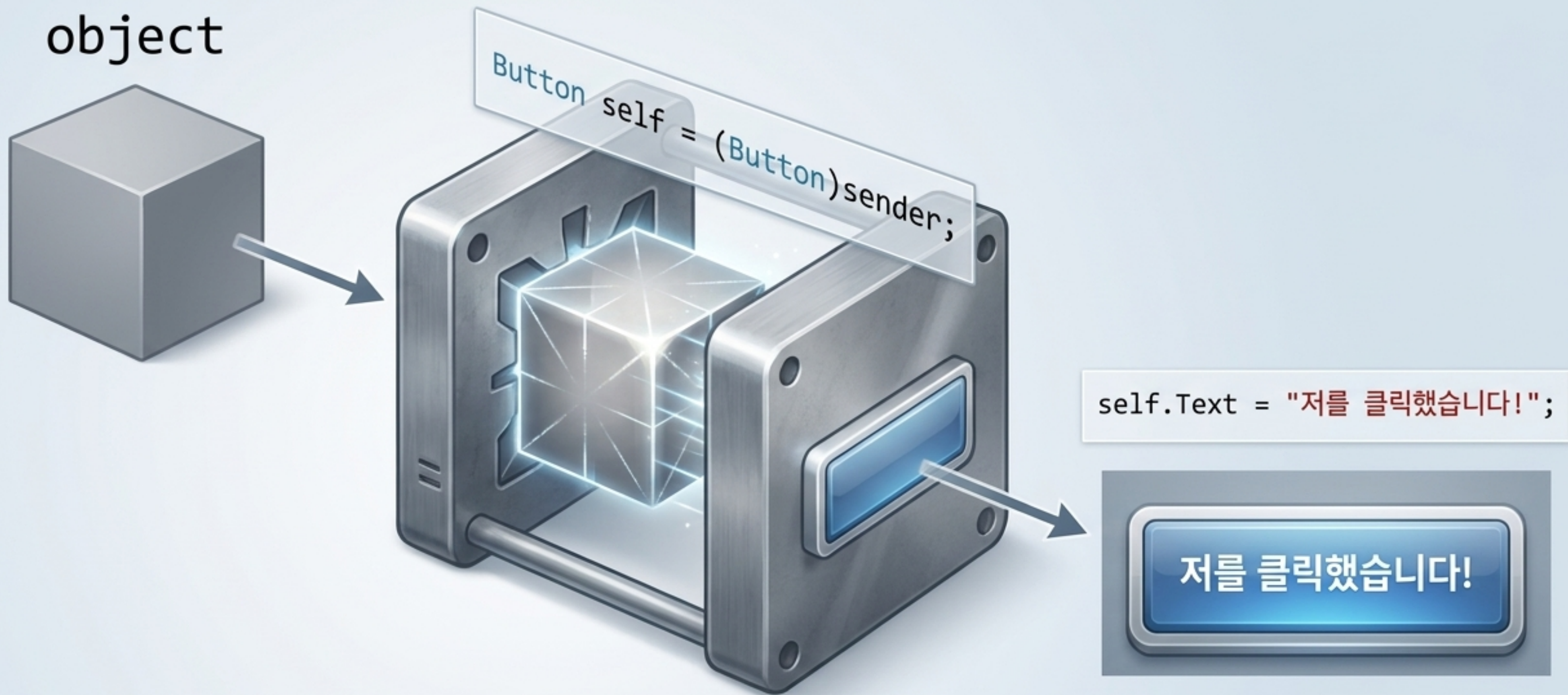
10 private void Button1_Click(object? sender, EventArgs e)
11 {
12     throw new NotImplementedException();
13 }

```

주의: 코드로 자동 완성된 메서드는 초기 상태로 NotImplementedException을 발생시킵니다. 내부 로직 구현을 잊지 마세요.



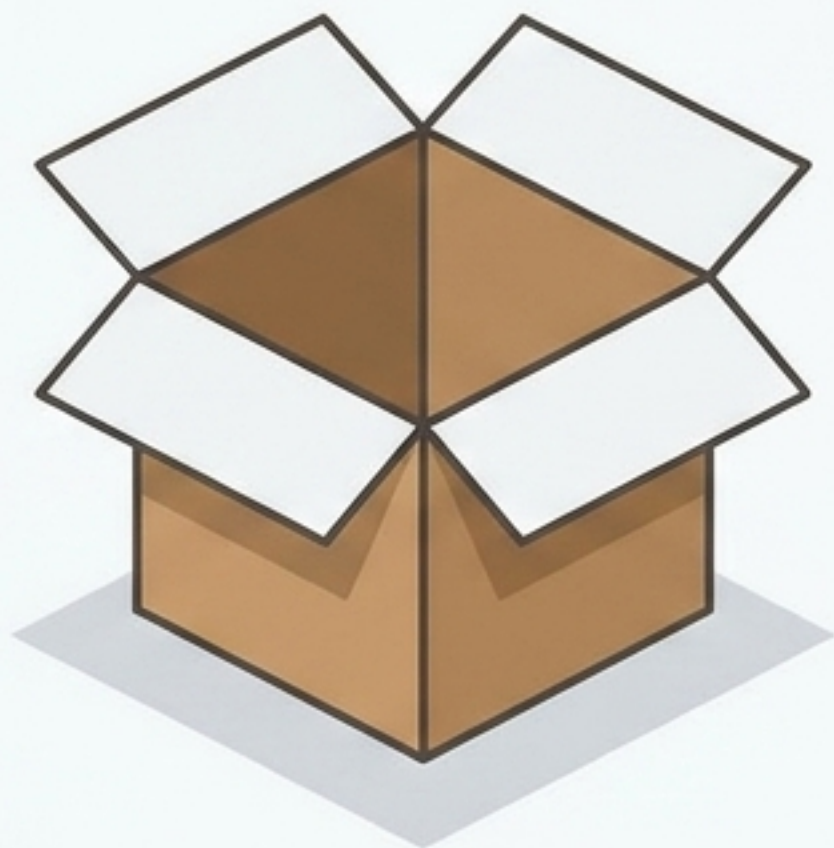
모든 이벤트 메서드는 이 두 가지 핵심 매개변수(Who & What)를 통해 상황을 완벽하게 파악합니다.



sender의 변신 (Casting)

sender는 단순한 object 껍데기입니다. 원래의 정체성(Button)으로 형변환(Casting)해야만 본연의 속성을 조작할 수 있습니다.

빈 상자 (단순 알림)



```
private void button1_Click(object  
sender, EventArgs e)  
{  
    e.  
}
```

- Equals
- ToString
- GetType
- GetHashCode

클릭처럼 ‘발생 사실’ 자체만 중요한 경우
추가 정보가 없습니다.

정보가 담긴 상자 (상세 보고서)

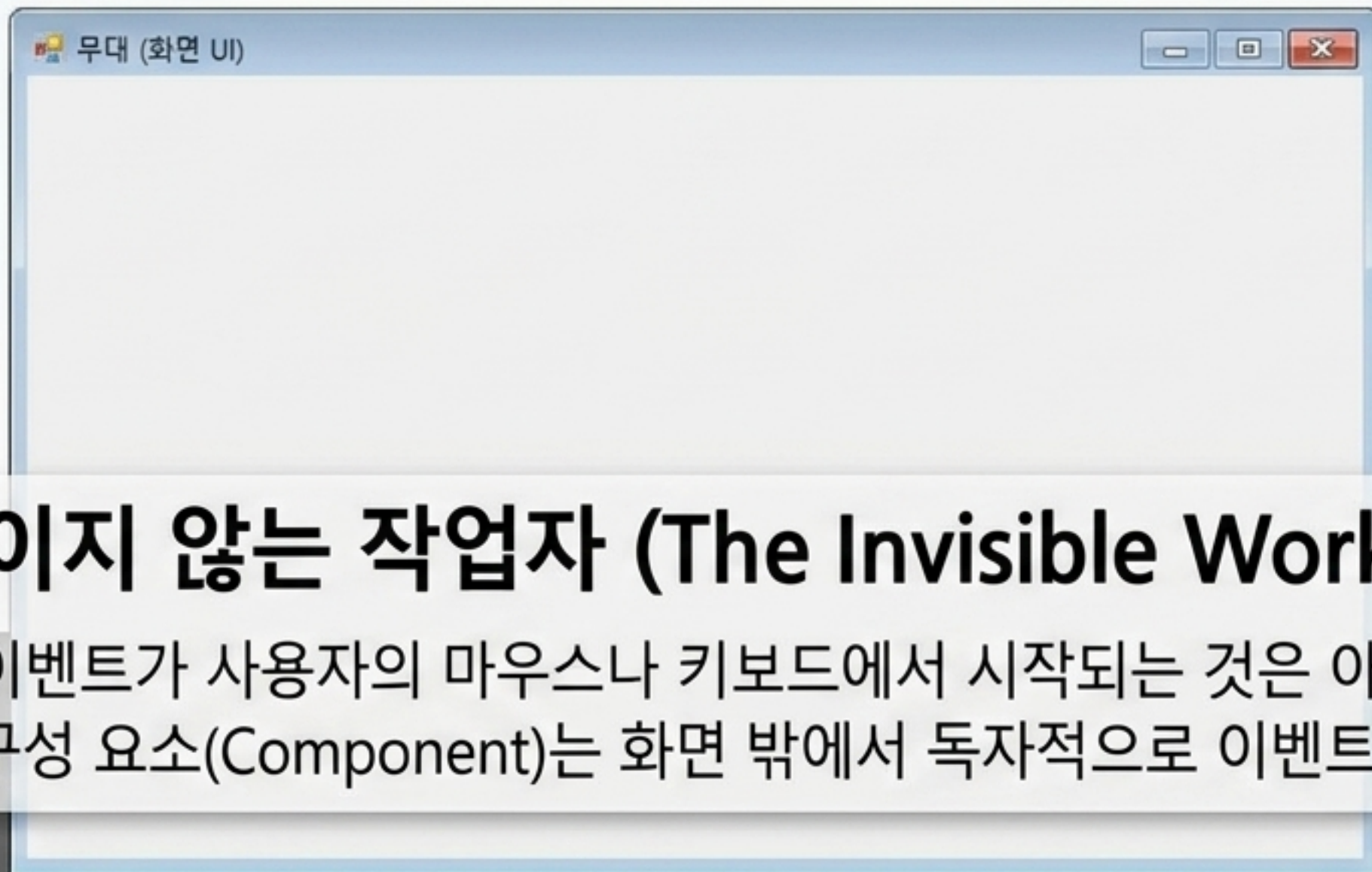


```
private void Form1_FormClosed(ob  
ject sender, FormClosedEventArgs  
{  
    e.  
}
```

- CloseReason
- Equals
- ToString
- GetType
- GetHashCode

폼 종료처럼 ‘왜 발생했는지(CloseReason)’ 맥락이
필수적인 이벤트는 구체적인 데이터를 함께 전달합니다.

무대
(Stage)



보이지 않는 작업자 (The Invisible Worker)

모든 이벤트가 사용자의 마우스나 키보드에서 시작되는 것은 아닙니다.
Timer와 같은 구성 요소(Component)는 화면 밖에서 독자적으로 이벤트를 발생시킵니다.

무대 뒤
(Component Tray)



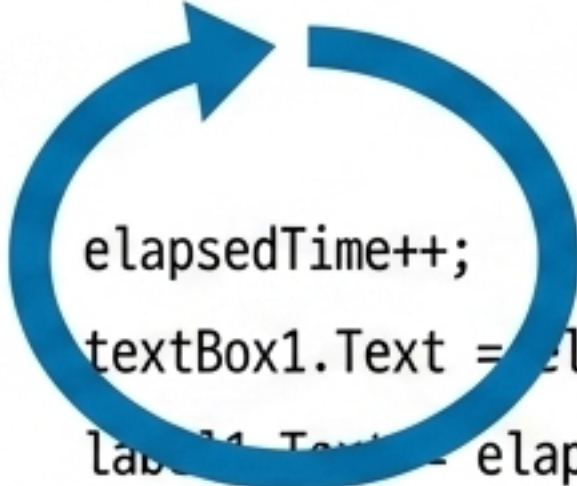


심장 박동: Tick 이벤트

Interval 속성에 정의된 시간(밀리초)마다 Timer는 사용자 개입 없이 스스로 무한히 Tick 이벤트를 트리거합니다.

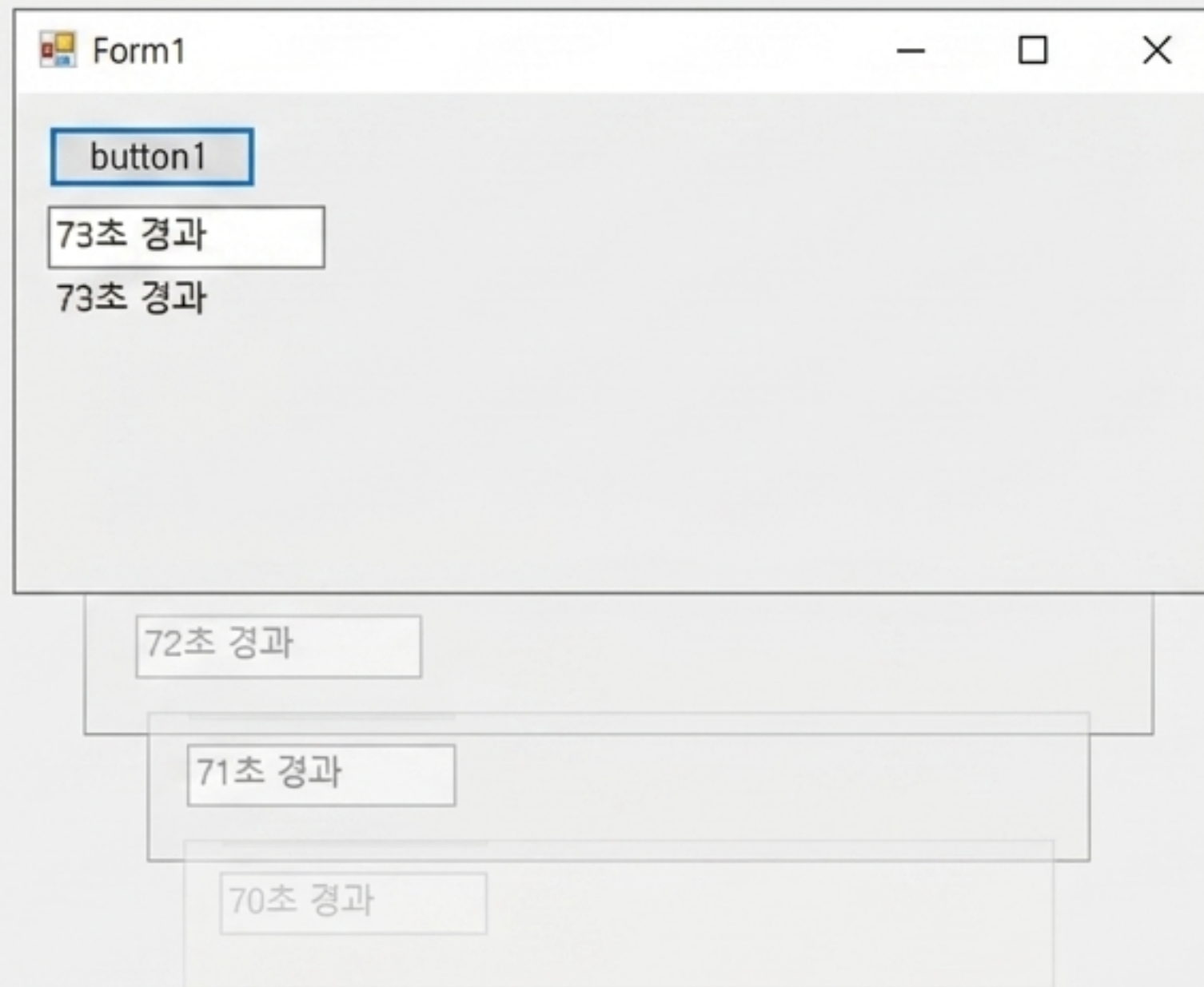
The Code Loop

```
01 private int elapsedTime = 0;  
02 private void timer1_Tick(object? sender, EventArgs e)  
03 {  
04  
05     elapsedTime++;  
05     textBox1.Text = elapsedTime + "초 경과";  
06     label1.Text = elapsedTime + "초 경과";  
07 }
```

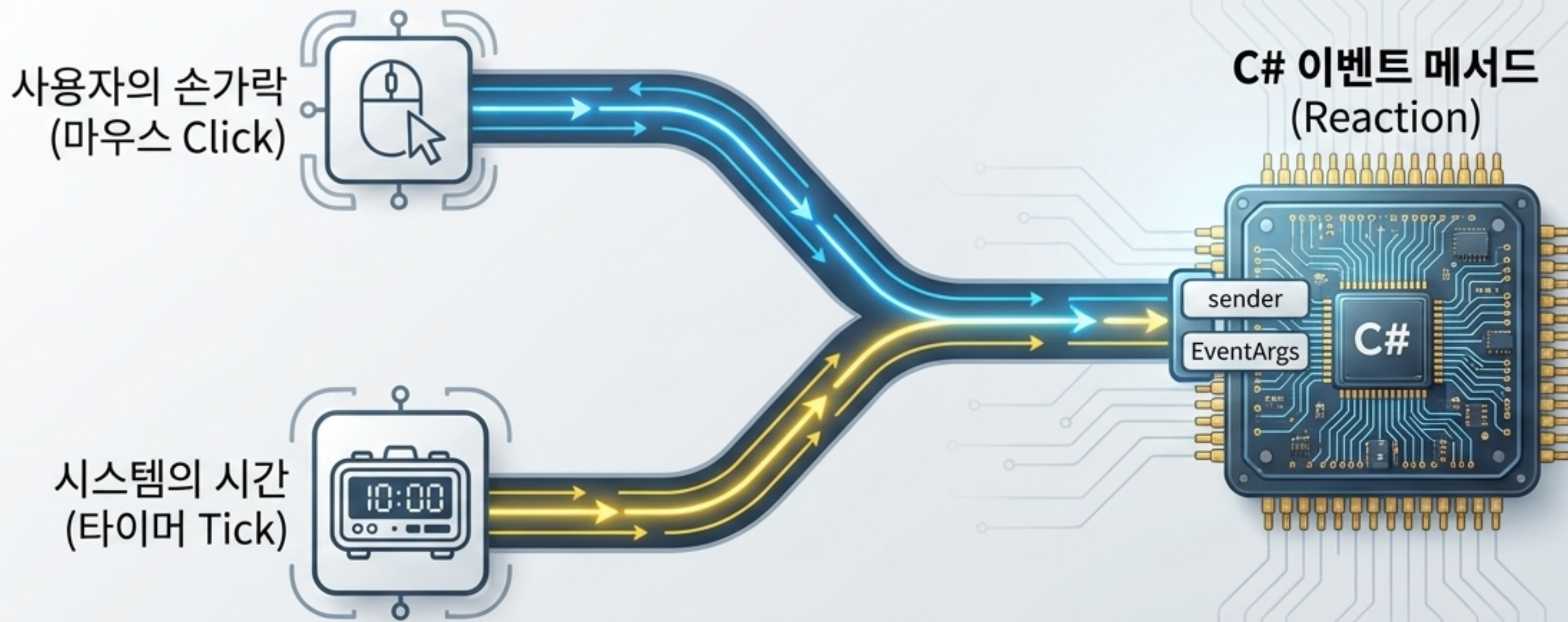


1초마다 무한 반복 증가

The Result



시간의 흐름(독립적 Action)이 화면 텍스트 업데이트(Reaction)로 이어지는 완벽한 자동화 회로가 완성되었습니다.



그 출발점이 사용자의 손가락이든, 시스템의 시간이든 관계없습니다.

윈도우 폼의 세계에서 모든 행동은 이벤트를 발생시키며, 우리는 C# 코드라는 정교한 배선을 통해 그 반응(Reaction)을 완벽하게 통제할 수 있습니다.